

Interfaces

Interfaces



Stony Brook University

Interfaces

- What is an interface?
 - An interface is a class-like construct that contains only constants and abstract methods.
- Why is an interface useful?
 - An interface is similar to an abstract class, but the intent of an interface is to **specify behavior** for objects.
 - For example: specify that the objects are **comparable, edible, cloneable, ...**
 - Allows multiple inheritance.

Constants in Java

- Use keyword: *final*

```
public class Cirkle {  
    // 'final' means each object must initialize its own value,  
    // and once set, it cannot be changed  
    private final double radius;  
  
    // 'static' means this variable is shared by all instances of the class  
    // All objects of Cirkle share the same value of PY  
    public static final double PY = 3.14;  
  
    // Constructor to initialize the radius for each object  
    public Cirkle(double radius) {  
        this.radius = radius;  
    }  
  
    public double getRadius() {return radius;}  
    // This setter is commented out because 'radius' is final  
    // Uncommenting this line would cause a compile-time error  
    public void setRadius(double radius) {  
        // this.radius = radius; // ❌ not allowed  
    }  
}
```

Define an Interface

Declaration:

```
public interface InterfaceName {  
    // constant declarations;  
    // method signatures;  
}
```

◦ Example:

```
public interface Edible {  
    /** Describe how to eat */  
    public abstract String howToEat();  
}
```

Interface is a Special Class

- An interface is treated like a special class in Java:
 - Each interface is compiled into a separate bytecode file, just like a regular class.
 - **Like an abstract class, you cannot create an instance from an interface using the new operator.**
 - Uses:
 - as a data type for a variable.
 - as the result of casting.

Interface Example

The **Edible** interface specifies whether an object is edible.

```
public interface Edible {  
    public abstract String howToEat();  
}
```

The class **Chicken** implements the **Edible** interface:

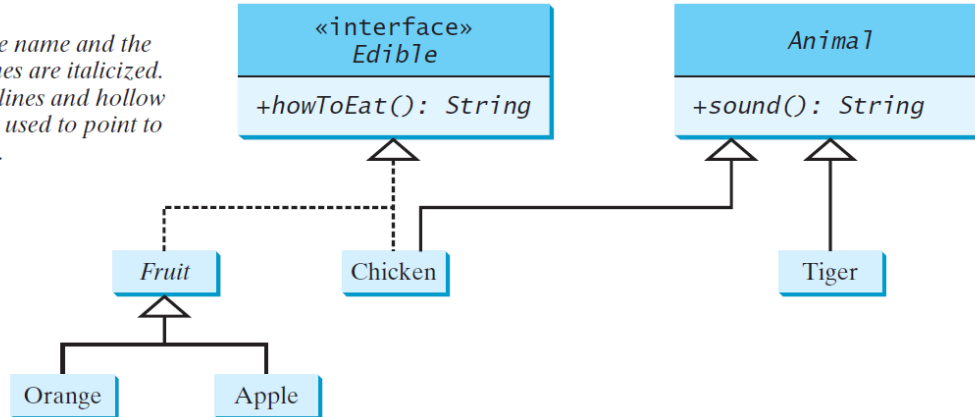
```
class Chicken extends Animal implements Edible {  
    public String howToEat() {  
        return "Chicken: Fry it.";  
    }  
}
```

Example: UML

You can now use the Edible interface to specify whether an object is edible. This is accomplished by letting the class for the object implement this interface using the ***implements*** keyword. For example, the classes Chicken and Fruit implement the Edible interface.

Notation:

The interface name and the method names are italicized. The dashed lines and hollow triangles are used to point to the interface.



Example: Code!

```
interface Edible {
    public abstract String howToEat(); /** Describe how to eat */
}
abstract class Animal {
    public abstract String sound();
}
class Chicken extends Animal implements Edible {
    public String howToEat() {
        return "Chicken: Fry it";
    }
    public String sound(){
        return "Chicken: cock-a-doodle-doo";
    }
}
class Tiger extends Animal { /** Does not implement Edible */
    public String sound(){
        return "Tiger: RROOAARR";
    }
}
abstract class Fruit implements Edible { }
class Apple extends Fruit {
    public String howToEat() {
        return "Apple: Make apple cider";
    }
}
class Orange extends Fruit {
    public String howToEat() {
        return "Orange: Make orange juice";
    }
}
```


Example cont..

```
public class TestEdible {  
    public static void main(String[] args) {  
        Object[] objects = {new Tiger(), new Chicken(), new Apple()};  
        for (int i = 0; i < objects.length; i++)  
            if (objects[i] instanceof Edible)  
                System.out.println(((Edible)objects[i]).howToEat());  
    }  
}
```

Abstract class Vs Interface

- We use **interfaces** to model capabilities shared by unrelated classes.
- We use **abstract classes** when classes are closely related and must share state, constructors, and common behavior.
- In our example, Edible is an interface because animals and fruits are unrelated, while Animal and Fruit are abstract classes because they represent base types with shared data (state) and behavior.

The Comparable Interface

```
// This interface is defined in java.lang package
public interface Comparable<E> {
    public int compareTo(E o) ;
}
```

- Comparable interface is a generic interface, generic type **E** is replaced by a concrete type.
- Generics enable types(classes and interfaces) to be parameters when defining classes, interfaces and methods.

Integer and BigInteger Classes

```
public interface Comparable<E> {  
    public int compareTo(E o);  
}
```

```
public class Integer extends Number  
    implements Comparable<Integer> {  
    // class body omitted  
  
    @Override  
    public int compareTo(Integer o) {  
        // Implementation omitted  
    }  
}
```

```
public class BigInteger extends Number  
    implements Comparable<BigInteger> {  
    // class body omitted  
  
    @Override  
    public int compareTo(BigInteger o) {  
        // Implementation omitted  
    }  
}
```

String and Date Classes

```
public interface Comparable<E> {  
    public int compareTo(E o);  
}
```

```
public class String extends Object  
    implements Comparable<String> {  
    // class body omitted  
  
    @Override  
    public int compareTo(String o) {  
        // Implementation omitted  
    }  
}
```

```
public class Date extends Object  
    implements Comparable<Date> {  
    // class body omitted  
  
    @Override  
    public int compareTo(Date o) {  
        // Implementation omitted  
    }  
}
```

Example: Cont..

- `System.out.println(new Integer(3).compareTo(new Integer(5)));`
- `System.out.println("ABC".compareTo("ABC"));`
- `java.util.Date date1 = new java.util.Date(2013, 1, 1);`
- `java.util.Date date2 = new java.util.Date(2012, 1, 1);`
- `System.out.println(date1.compareTo(date2));`

Generic sort Method

Let **n** be an **Integer** object, **s** be a **String** object, and **d** be a **Date** object. All the following expressions are **true**.

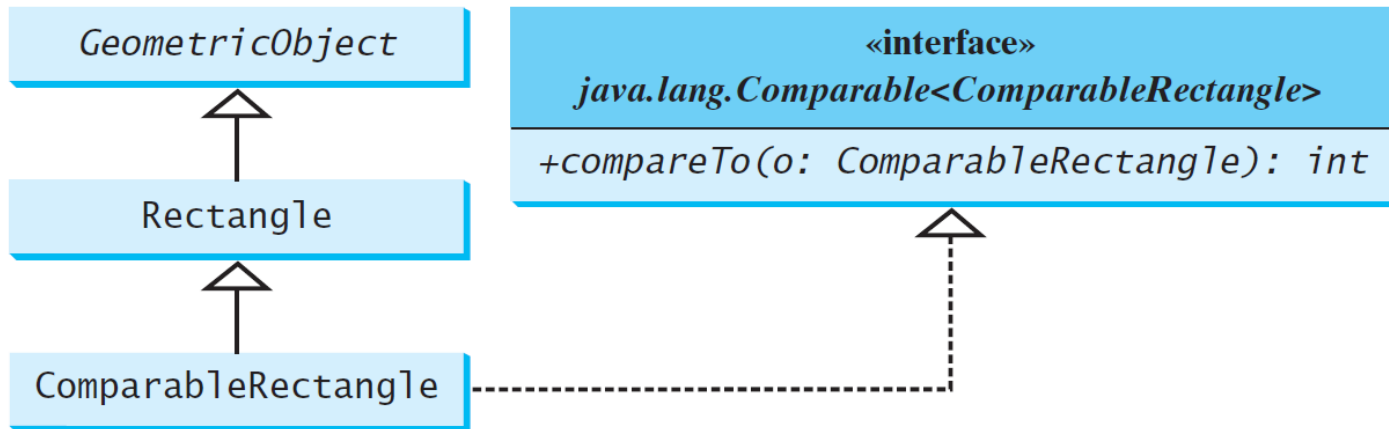
```
n instanceof Integer
n instanceof Object
n instanceof Comparable
```

```
s instanceof String
s instanceof Object
s instanceof Comparable
```

```
d instanceof java.util.Date
d instanceof Object
d instanceof Comparable
```

The `java.util.Arrays.sort(array)` method requires that the elements in an array are instances of `Comparable<E>`.

Defining Classes to Implement Comparable



Code: Classes to Implement Comparable

```
public interface Comparable<E> {  
    public int compareTo(E o);  
}
```

```
public class ComparableRectangle extends Rectangle  
    implements Comparable<ComparableRectangle> {  
    /** Construct a ComparableRectangle with specified properties */  
    public ComparableRectangle(double width, double height) {  
        super(width, height);  
    }  
  
    /** Implement the compareTo method defined in Comparable */  
    public int compareTo(ComparableRectangle o) {  
        if (getArea() > o.getArea())  
            return 1;  
        else if (getArea() < o.getArea())  
            return -1;  
        else  
            return 0;  
    }  
}
```

The Cloneable Interfaces

- ***Marker Interface***: An empty interface.
- A marker interface ***does not contain constants*** or ***methods***. It is used to denote that a ***class possesses certain desirable properties***.
- A ***class that implements the Cloneable interface is marked cloneable***, and its objects can be cloned using the clone() method defined in the Object class.

Cloneable contd..

- The ***Cloneable*** interface *in java.lang* package is defined as below:

```
package java.lang;  
public interface Cloneable{  
}
```

- The clone method as defined in Object class is:
protected native Object clone() **throws** CloneNotSupportedException;

Contd..

- The keyword ***native*** indicates that the method is not written in Java, but is implemented in the JVM for native platform.
- The method's code is written in a **lower-level language** such as **C or C++**
- The code is part of the **JVM (Java Virtual Machine)**, not part of your Java program



Stony Brook
University